Mapeamento Objeto-Relacional — Plataforma Nexo

Persistência | Aquiles Maciel Amaral

1. Objetivo

Este documento registra o mapeamento entre o diagrama de classes de domínio e o Modelo Relacional, estabelecendo a correspondência entre classe e tabela, atributo e coluna, associação e chave estrangeira, multiplicidade e cardinalidade, e o tratamento dado à ausência de herança no domínio.

2. Convenções

- Nomes de classe seguem a grafia do diagrama de classes de domínio; nomes de tabela seguem o Modelo Relacional (snake_case minúsculo).
- A coluna "Tipo SQL" usa os tipos já definidos no Dicionário de Dados.
- Toda classe recebe uma coluna `id` (INT, chave primária, autoincremento) não declarada explicitamente como atributo UML é a implementação da identidade do objeto, exigência do modelo relacional e não um atributo do domínio.

3. Correspondência classe ↔ tabela

3.1 Acesso e permissões

Classe Usuário → tabela usuario

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
nome: String	nome	VARCHAR(150)	—	—
email: String	email	VARCHAR(150)	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_perfil_acesso	INT	"possui perfil" (0..*, 1) com Perfil de Acesso	FK → perfil_acesso.id, N para 1

Classe Perfil de Acesso → tabela perfil_acesso

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id (implícito)	id	INT	—	PK
nome: String	nome	VARCHAR(30)	—	—
—	—	—	"autoriza" (0..*, 0..*) com Permissão	N para N, resolvida pela tabela perfil_permissao

Classe Permissão → tabela permissao

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id (implícito)	id	INT	—	PK
recurso: String	recurso	VARCHAR(80)	—	—
operação: String	operacao	VARCHAR(30)	—	—
condições: String	condicoes	VARCHAR(255)	—	—

Associação N:N "autoriza" → tabela perfil_permissao

A associação entre Perfil de Acesso e Permissão não corresponde a uma classe do diagrama de domínio; é mapeada como tabela associativa, técnica padrão para relações N:N em modelo relacional.

Coluna	Tipo SQL	Origem
id	INT	Chave técnica da tabela associativa (sem equivalente UML)
id_perfil	INT	FK → perfil_acesso.id (lado "1" da associação, a partir de Perfil de Acesso)
id_permissao	INT	FK → permissao.id (lado "1" da associação, a partir de Permissão)

3.2 Pessoas e vínculos

Classe Paciente → tabela paciente

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
nome: String	nome	VARCHAR(150)	—	—
dadosCadastrais: String	dados_cadastrais	VARCHAR(500)	—	—
—	id_usuario	INT	"identidade de acesso" (0..1, 1) com Usuário	FK → usuario.id, aceita nulo, 0..1 para 1

Classe Psicólogo → tabela psicologo

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
nome: String	nome	VARCHAR(150)	—	—

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
identificaçãoProfissional: String	identificacao_prof	VARCHAR(20)	—	—
especialidades: String [0..*]	especialidades	VARCHAR(300)	—	Multivalorado mapeado como campo único serializado (ver seção 5)
apresentação: String	apresentacao	VARCHAR(500)	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_usuario	INT	"identidade de acesso" (0..1, 1) com Usuário	FK → usuario.id, aceita nulo, 0..1 para 1

Classe Vínculo de Responsabilidade → tabela vinculo_responsabilidade

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
situação: String	situacao	VARCHAR(20)	—	—
—	id_usuario_responsavel	INT	"responsável" (0..*, 1) com Usuário	FK → usuario.id, N para 1
—	id_paciente_dependente	INT	"dependente" (0..*, 1) com Paciente	FK → paciente.id, N para 1

Nota sobre operações de classe não mapeadas: as operações `atualizarCadastro()`, `atualizarInformações()`, `éVálido()` e `permite()` descritas no diagrama de classes são comportamento de aplicação, não persistência; não há coluna correspondente, conforme item 5 (casos sem correspondência).

3.3 Agenda e agendamentos

Classe Disponibilidade → tabela disponibilidade

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
diaDaSemana: Integer	dia_da_semana	INT	—	—

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
horaInicial: Time	hora_inicial	TIME	—	—
horaFinal: Time	hora_final	TIME	—	—
vigênciaInicial: Date	vigencia_inicial	DATE	—	—
vigênciaFinal: Date [0..1]	vigencia_final	DATE	—	aceita nulo (opcional já no UML)
—	id_psicologo	INT	"disponibilidade de" (0..*, 1) com Psicólogo	FK → psicologo.id, N para 1

Classe Bloqueio de Agenda → tabela bloqueio_agenda

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
início: DateTime	inicio	DATETIME	—	—
fim: DateTime	fim	DATETIME	—	—
motivo: String [0..1]	motivo	VARCHAR(200)	—	aceita nulo
—	id_psicologo	INT	"bloqueia agenda de" (0..*, 1) com Psicólogo	FK → psicologo.id, N para 1

Classe Agendamento → tabela agendamento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
início: DateTime	inicio	DATETIME	—	—
fim: DateTime	fim	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_psicologo	INT	"profissional" (0..*, 1) com Psicólogo	FK → psicologo.id, N para 1
—	id_paciente	INT	"consulta marcada para"	FK → paciente.id, N

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
			(0..*, 1) com Paciente	para 1
—	id_recorrencia	INT	"ocorrência de" (0..*, 0..1) com Recorrência	FK → recorrencia.id, aceita nulo, N para 0..1

3.4 Atendimento e evolução clínica

Classe Atendimento → tabela atendimento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
data: DateTime	data	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_agendamento	INT	"realiza consulta marcada" (0..1, 1) com Agendamento	FK → agendamento.id, UQ, 0..1 para 1

Classe Evolução Clínica → tabela evolucao_clinica

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
conteúdo: String	conteudo	VARCHAR(2000)	—	—
dataRegistro: DateTime	data_registro	DATETIME	—	—
históricoVersões: String [0..*]	historico_versoes	VARCHAR(2000)	—	Multivalorado mapeado como campo único serializado (ver seção 5)
—	id_atendimento	INT	"registra evolução de" (0..*, 1) com Atendimento	FK → atendimento.id, N para 1
—	id_psicologo_autor	INT	"autor profissional" (0..*, 1) com	FK → psicologo.id, N para 1

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
			Psicólogo	

3.5 Recorrência

Classe Recorrência → tabela recorrencia

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
periodicidade: String	periodicidade	VARCHAR(20)	—	—
horárioReservado: Time	horario_reservado	TIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
condições: String	condicoes	VARCHAR(300)	—	aceita nulo
—	id_paciente	INT	"acordo do paciente" (0..*, 1) com Paciente	FK → paciente.id, N para 1
—	id_psicologo	INT	"acordo com profissional" (0..*, 1) com Psicólogo	FK → psicologo.id, N para 1

Nota: a associação "materializa ocorrências" (0..1, 0..*) entre Recorrência e Agendamento é mapeada pela FK id_recorrencia já declarada em agendamento (seção 3.3), evitando redundância de chave estrangeira em duas tabelas para a mesma relação.

3.6 Cobranças e pagamentos

Classe Cobrança → tabela cobranca

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
valor: Decimal	valor	FLOAT	—	—
vencimento: Date	vencimento	DATE	—	—
modalidade: String	modalidade	VARCHAR(20)	—	—

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
periodoReferência: String [0..1]	periodo_referencia	VARCHAR(20)	—	aceita nulo
situação: String	situacao	VARCHAR(20)	—	—
—	id_paciente	INT	"obrigação do paciente" (0..*, 1) com Paciente	FK → paciente.id, N para 1
—	id_agendamento	INT	"consulta avulsa" (0..*, 0..1) com Agendamento	FK → agendamento.id, aceita nulo, N para 0..1
—	id_recorrencia	INT	"mensalidade" (0..*, 0..1) com Recorrência	FK → recorrencia.id, aceita nulo, N para 0..1

Classe Tentativa de Pagamento → tabela tentativa_pagamento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
modalidade: String	modalidade	VARCHAR(20)	—	—
referênciaExterna: String	referencia_externa	VARCHAR(100)	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_cobranca	INT	"tentativas de quitação" (0..*, 1) com Cobrança	FK → cobranca.id, N para 1

Classe Pagamento → tabela pagamento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
valorConfirmado: Decimal	valor_confirmado	FLOAT	—	—
modalidade: String	modalidade	VARCHAR(20)	—	—

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
dataConfirmação: DateTime	data_confirmacao	DATETIME	—	—
referênciaTransação: String	referencia_transacao	VARCHAR(100)	—	—
—	id_tentativa_pagamento	INT	"resulta em confirmação" (0..1, 1) com Tentativa de Pagamento	FK → tentativa_pagamento.id, UQ, 0..1 para 1

3.7 Lista de espera e encaixes

Classe Lista de Espera → tabela lista_espera

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
—	id_psicologo	INT	"lista do profissional" (0..1, 1) com Psicólogo	FK → psicologo.id, UQ, 0..1 para 1

Classe Inscrição na Lista de Espera → tabela inscricao_lista_espera

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
preferências: String	preferencias	VARCHAR(300)	—	aceita nulo
dataInclusão: DateTime	data_inclusao	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_lista_espera	INT	"contém inscrições" (0..*, 1) com Lista de Espera	FK → lista_espera.id, N para 1
—	id_paciente	INT	"participante" (0..*, 1) com Paciente	FK → paciente.id, N para 1

Classe Oferta de Encaixe → tabela oferta_encaixe

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
início: DateTime	inicio	DATETIME	—	—
fim: DateTime	fim	DATETIME	—	—
prazoResposta: DateTime	prazo_resposta	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_inscricao	INT	"recebe ofertas" (0..*, 1) com Inscrição na Lista de Espera	FK → inscricao_lista_espera.id, N para 1
—	id_agendamento_gerado	INT	"gera após aceitação" (0..1, 0..1) com Agendamento	FK → agendamento.id, aceita nulo, UQ, 0..1 para 0..1

3.8 Documentos e compartilhamentos

Classe Documento → tabela documento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
tipo: String	tipo	VARCHAR(50)	—	—
referênciaConteúdo: String	referencia_conteudo	VARCHAR(255)	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_paciente	INT	"documento do paciente" (0..*, 1) com Paciente	FK → paciente.id, N para 1
—	id_psicologo	INT	"profissional responsável" (0..*, 1) com Psicólogo	FK → psicologo.id, N para 1

Classe Compartilhamento de Documento → tabela compartilhamento_documento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
finalidade: String	finalidade	VARCHAR(200)	—	—
fundamentoAutorização: String	fundamento_autorizacao	VARCHAR(200)	—	—
validade: DateTime	validade	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
—	id_documento	INT	"disponibilização autorizada" (0..*, 1) com Documento	FK → documento.id, N para 1

Classe Destinatário de Compartilhamento → tabela destinatario_compartilhamento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
identificação: String	identificacao	VARCHAR(150)	—	—
dadosVerificação: String	dados_verificacao	VARCHAR(200)	—	—
—	id_compartilhamento	INT	"destinatário específico" (0..*, 1) com Compartilhamento de Documento	FK → compartilhamento_documento.id, N para 1

Classe Registro de Acesso ao Documento → tabela registro_acesso_documento

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
data: DateTime	data	DATETIME	—	—
situação: String	situacao	VARCHAR(20)	—	—
identificaçãoAcesso: String [0..1]	identificacao_acesso	VARCHAR(150)	—	aceita nulo (opcional já no UML)

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
—	id_compartilhamento	INT	"histórico de acessos" (0..*, 1) com Compartilhamento de Documento	FK → compartilhamento_documento.id, N para 1

3.9 Comunicação e auditoria

Classe Notificação → tabela notificacao

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
tipo: String	tipo	VARCHAR(40)	—	—
conteúdoPermitido: String	conteudo_permitido	VARCHAR(500)	—	—
data: DateTime	data	DATETIME	—	—
situaçãoEnvio: String	situacao_envio	VARCHAR(20)	—	—
—	id_usuario_destinatario	INT	"destinatário" (0..*, 1) com Usuário	FK → usuario.id, N para 1
—	id_oferta_encaixe	INT	"contexto da oferta" (0..*, 0..1) com Oferta de Encaixe	FK → oferta_encaixe.id, aceita nulo, N para 0..1
—	id_compartilhamento	INT	"contexto do compartilhamento" (0..*, 0..1) com Compartilhamento de Documento	FK → compartilhamento_documento.id, aceita nulo, N para 0..1

Classe Registro de Auditoria → tabela registro_auditoria

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
id: UUID	id	INT	—	PK
ação: String	acao	VARCHAR(80)	—	—

Atributo UML	Coluna	Tipo SQL	Associação UML	FK / cardinalidade
data: DateTime	data	DATETIME	—	—
recursoAfetado: String	recurso_afetado	VARCHAR(100)	—	—
resultado: String	resultado	VARCHAR(20)	—	—
—	id_usuario_autor	INT	"autor da operação" (0..*, 0..1) com Usuário	FK → usuario.id, aceita nulo, N para 0..1

4. Tratamento de herança

O diagrama de classes de domínio não utiliza herança entre as classes do Nexo. O documento oficial registra explicitamente: "Não foi preciso generalizar Paciente, Psicólogo e Usuário. Compartilhar dados pessoais não basta para justificar herança" e "Herança: Não foi preciso generalizar Paciente, Psicólogo e Usuário".

Em consequência, não há regra de mapeamento de herança (tabela única, tabela por subclasse ou tabela por classe concreta) a aplicar neste projeto: cada classe do domínio corresponde a exatamente uma tabela independente, sem relação de superclasse/subclasse entre elas. As associações que, à primeira vista, poderiam sugerir uma hierarquia — Usuário com Paciente, Usuário com Psicólogo são, no diagrama de classes, associações comuns de multiplicidade opcional (0..1 para 1), mapeadas como chave estrangeira opcional (`paciente.id_usuario`, `psicologo.id_usuario`), não como chave primária compartilhada de uma relação de generalização.

O único ponto em que a ausência de herança exigiu uma decisão de modelagem foi o tratamento do perfil do usuário: em vez de subclasses de Usuário (UsuárioAdministrador, UsuárioPsicólogo etc.), o diagrama de domínio usa uma associação de Usuário com Perfil de Acesso, mapeada como chave estrangeira `usuario.id_perfil_acesso,` uma solução de modelagem que simula a diferenciação de papéis sem recorrer a herança, compatível com a nota do próprio diagrama de domínio de que "existem exatamente quatro perfis fixos".

5. Casos sem correspondência direta

Alguns elementos do diagrama de classes de domínio não têm equivalente direto no modelo relacional, por não serem dados persistidos ou por exigirem tratamento especial:

- **Operações de classe** (`atualizarCadastro()`, `registrarConfirmação()`, `cancelar()`, `encerrar()`, `éVálido()`, `permite()`, entre outras): representam comportamento, implementado na camada de aplicação, não no banco de dados. Não há coluna correspondente a uma operação.
- **Atributos multivalorados** (`especialidades: String [0..*]` em Psicólogo; `históricoVersões: String [0..*]` em Evolução Clínica): o diagrama de classes permite

múltiplos valores no mesmo atributo; o modelo relacional os representa como um único campo VARCHAR com valores serializados, uma desnormalização deliberada e justificada no documento de Normalização, em vez de criar tabelas filhas específicas para cada lista.

- **Identificadores do tipo UUID**: o diagrama de classes declara `id: UUID` em todas as classes; o modelo relacional usa `INT` autoincremento como chave primária técnica. A escolha segue a convenção adotada desde o DER lógico e o Modelo Relacional, priorizando simplicidade de geração e indexação sobre a fidelidade literal ao tipo UUID do diagrama de classes; não há perda de informação de domínio, apenas uma escolha de implementação da identidade do objeto.
- **Serviços e componentes externos**: o diagrama de domínio e as justificativas mencionam explicitamente que "autenticação e persistência pertencem a outros componentes" e que "envio, captura de eventos, integrações e persistência são responsabilidades externas às entidades". Não há, portanto, tabela correspondente a esses componentes (serviço de autenticação, serviço de envio de notificação, gateway de pagamento) eles são integrações externas referenciadas por colunas como `tentativa_pagamento.referencia_externa` e `pagamento.referencia_transacao`, não entidades do modelo relacional.

6. Observações finais

As 25 tabelas do Modelo Relacional correspondem às 25 classes do diagrama de domínio, uma a uma, sem necessidade de desdobrar nenhuma classe em mais de uma tabela nem de agrupar mais de uma classe em uma única tabela. As duas tabelas associativas do modelo (`perfil_permissao`, resolvendo a associação N:N "autoriza"; e as FKs de `vinculo_responsabilidade`, que já nasce como classe associativa no próprio diagrama de domínio) não têm classe própria no sentido de "classe candidata de negócio", mas sim de implementação de associação — tratamento já previsto no checklist deste cartão ("associações mapeadas em chaves estrangeiras").